Information Fusion 115 (2025) 102748

Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus

Full length article

Vul-LMGNNs: Fusing language models and online-distilled graph neural networks for code vulnerability detection☆

Ruitong Liu [a,d], Yanbin Wang [a,b,c,*], Haitao Xu [c,*], Jianguo Sun [b,*], Fan Zhang [c], Peiyue Li [d], Zhenhao Guo [c]

[a] Shenzhen MSU-BIT University, Shenzhen, 518172, China
[b] Hangzhou Research Institute of Xidian University, 310027, Hangzhou, China
[c] School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University, Hangzhou, 330000, China
[d] People's Public Security University of China, Beijing, 100038, China

ARTICLE INFO

Dataset link: https://github.com/Vul-LMGNNs/vul-LMGGNN

Keywords:
Code vulnerability detection
Graph information fusion
Pre-trained code model
Joint training

ABSTRACT

Code Language Models (codeLMs) and Graph Neural Networks (GNNs) are widely used in code vulnerability detection. However, a critical yet often overlooked issue is that GNNs primarily rely on aggregating information from adjacent nodes, limiting structural information transfer to single-layer updates. In code graphs, nodes and relationships typically require cross-layer information propagation to fully capture complex program logic and potential vulnerability patterns. Furthermore, while some studies utilize codeLMs to supplement GNNs with code semantic information, existing integration methods have not fully explored the potential of their collaborative effects.

To address these challenges, we introduce Vul-LMGNNs that integrates pre-trained CodeLMs with GNNs, leveraging knowledge distillation to facilitate cross-layer propagation of both code semantic knowledge and structural information. Specifically, Vul-LMGNNs utilizes Code Property Graphs (CPGs) to incorporate code syntax, control flow, and data dependencies, while employing gated GNNs to extract structural information in the CPG. To achieve cross-layer information transmission, we implement an online knowledge distillation (KD) program that enables a single student GNN to acquire structural information extracted from a simultaneously trained counterpart through an alternating training procedure. Additionally, we leverage pre-trained CodeLMs to extract semantic features from code sequences. Finally, we propose an "implicit-explicit" joint training framework to better leverage the strengths of both CodeLMs and GNNs. In the implicit phase, we utilize CodeLMs to initialize the node embeddings of each student GNN. Through online knowledge distillation, we facilitate the propagation of both code semantics and structural information across layers. In the explicit phase, we perform linear interpolation between the CodeLM and the distilled GNN to learn a late fusion model. The proposed method, evaluated across four real-world vulnerability datasets, demonstrated superior performance compared to 17 state-of-the-art approaches. Our source code can be accessed via GitHub: https://github.com/Vul-LMGNN/vul-LMGGNN.

1. Introduction

With the rapid expansion of the open-source community, software vulnerability detection technology has become a significant concern in the software industry and cybersecurity domain. Vulnerabilities pose a threat to the integrity and availability of software and computer systems, potentially leading to privilege escalation, leakage of sensitive data, denial of service, and various other attacks, resulting in substantial economic and societal losses [1,2]. Traditionally, developers and security engineers rely on rule-based code analysis and symbolic execution techniques [3]. Although these methods can detect vulnerabilities effectively, they often generate high false-positive rates, requiring extensive manual verification.

To improve the efficiency of code vulnerability detection, extensive research has leveraged deep learning (DL) models for automated vulnerability detection. These methods extract features from the source

☆ This work was supported in part by the National Natural Science Foundation of China under Grant 61902342, and in part by the Defense Industrial Technology Development Program, China under Grant JCKY 2021602B002.

* Corresponding authors.
E-mail addresses: liuruitong@bupt.edu.cn (R. Liu), wyb@zuaa.zju.edu.cn (Y. Wang), haitaoxu@zju.edu.cn (H. Xu), jgsun@xidian.edu.cn (J. Sun), fanzhang@zju.edu.cn (F. Zhang), lipeiyue@ppsuc.edu.cn (P. Li), guozhenhao17@mails.ucas.ac.cn (Z. Guo).

https://doi.org/10.1016/j.inffus.2024.102748
Received 2 September 2024; Received in revised form 4 October 2024; Accepted 15 October 2024
Available online 21 October 2024

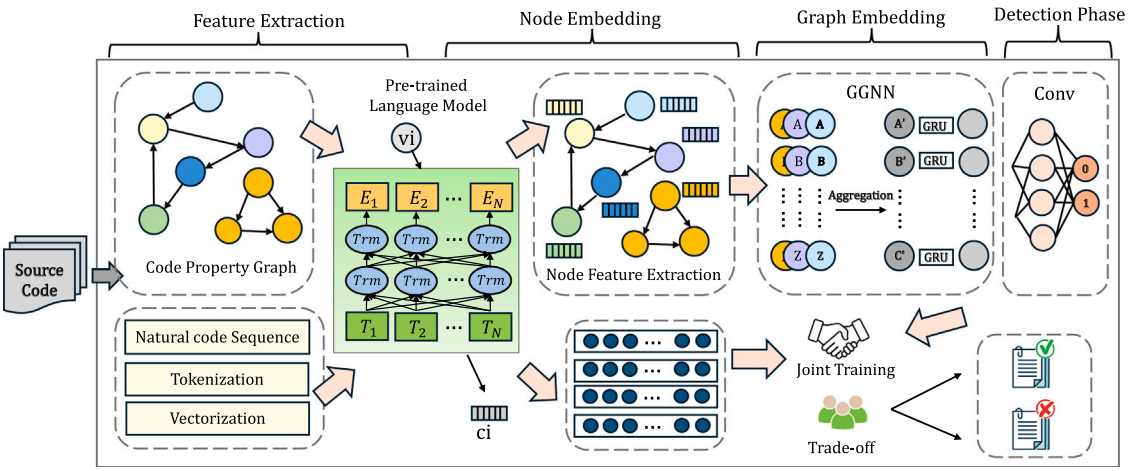

Fig. 1. Overview of the Vul-LMGNNs vulnerability detection framework.

code to generate initial embedding vectors, which are then fed into neural networks to learn vulnerability patterns and produce classification results, thereby achieving automatic detection capabilities [4].

Deep learning for code vulnerability detection primarily falls into two categories: sequence-based and graph-based approaches. Sequence-based methods convert source code or its structures into serialized forms, treating elements as tokens [5,6]. These approaches employ various architectures such as LSTMs [7,8], CNNs [9], or modern language models to detect vulnerabilities by learning sequential features. Graph-based methods, conversely, transform source code into heterogeneous graph structures like Abstract Syntax Trees (AST), Control Flow Graphs (CFG), and Program Dependence Graphs (PDG). These methods then employ Graph Neural Networks (GNNs) to capture both local structures and dependencies, offering richer syntactic and structural insights compared to code sequences [7,10].

However, current work has two main limitations: First, current GNN-based methods for vulnerability detection suffer from a common weakness: the GNNs they employ are inherently limited in their ability to propagate structural knowledge across multiple layers of the graph. This limitation stems from the fundamental architecture of typical GNNs, which primarily aggregate information within a single layer during each iteration of the message-passing process. Even in multi-layer GNNs, where multiple rounds of aggregation are performed, each layer's updates are still based on immediate neighbors, failing to directly capture long-range dependencies or complex structural patterns that span across multiple layers of the graph. This localized information processing can be particularly problematic in the context of code analysis, where vulnerabilities often arise from intricate interactions between distant parts of the code structure, requiring a more holistic understanding of the program's architecture and data flow. Secondly, GNNs offer distinct strengths in code analysis. Sequence-based approaches excel at capturing contextual semantics, while GNNs provide valuable structural insights. However, current research rarely leverages the potential synergy between these complementary approaches.

To address these challenges, we propose Vul-LMGNNs, a joint training framework that effectively integrates codeLMs and GNNs, while resolving information transfer limitations in code graph learning. The core of Vul-LMGNNs is a two-stage fusion scheme implementing an "implicit-explicit" integration: (1) Implicit Stage: We utilize semantic embeddings extracted by codeLMs to initialize nodes in the code GNN. This approach fuses semantic information with code structural information during graph update learning, constituting the initial information integration. (2) Explicit Stage: We employ linear interpolation to combine the predictions of both model types, achieving the final model fusion. Furthermore, by integrating online knowledge distillation into the first stage of our fusion architecture, we simultaneously facilitate the effective cross-layer information propagation of both semantic and graph structural information.

The contributions of our work can be summarized as follows:

- We propose a novel approach for detecting code vulnerabilities that leverages codeLMs to learn the semantics from source code sequences and employs gated GNN to grasp syntactic, control, and data dependencies from code attribute graphs.
- We propose an "implicit-explicit" dual-learning framework for the systematic integration of CodeLMs and GNNs. In the implicit stage, we perform an embedding-level fusion of the two models. In the explicit stage, we employ linear interpolation for the output consolidation at the model endpoints.
- We propose the incorporation of an online knowledge distillation module into the joint training framework of CodeLMs and GNNs to address the limitations of cross-layer knowledge propagation in code graphs and to further facilitate the integration of code semantic information with code structural information.
- The proposed method is extensively evaluated on four public datasets, achieving state-of-the-art performance, with approximately a 10% improvement in F1 scores on two challenging imbalanced datasets compared to the previous most competitive baseline methods.

The rest of this paper is structured as follows: Section 2 provides an overview of the background and related work. Section 4 outlines the composition of the dataset. Section 3 delves into the design specifics of our model. Section 5 presents the experimental outcomes and evaluates our model's performance relative to the baseline method across datasets. Finally, Section 6 summarizes the paper and outlines directions for future research.

2. Related work

In this section, we review the most relevant works to our study, focusing on those based on deep learning techniques. These can be broadly categorized into two groups: sequence-based approaches and graph-based approaches.

2.1. Sequence based models

Current studies based on deep sequence models generally follow the process of preprocessing, vectorization, and neural network modeling [4,5,11–13]. In data preprocessing, the raw source code is subjected to slicing and normalization techniques, after which it is parsed into a sequence of tokens. Subsequently, these tokens are transformed into

vectors suitable for neural network processing. RNN and transformer-based models are used to learn contextual information within token sequences and to make the final defect prediction. RNN-based works, such as VulDeePecker [8] and SySeVR [14], have introduced lexical analysis, which converts the source code into a more fine-grained code snippet. A potential concern with code slicing is that the extracted code representations may not encompass all vulnerable code snippets. On the contrary, transformer-based methods utilize token vectorization techniques that extract more vulnerability-aware features. Transformer-based approaches often omit code slicing and normalization strategies, opting instead to directly tokenize the source code. Chen et al. evaluated the performance of several Transformer-based methods, including CodeBERT [15] and GraphCodeBERT [16], on the DiverseVul dataset for vulnerability detection. Other methods have adopted different tokenization strategies from the NLP domain; for instance, CodeT5 [17] uses byte-level byte-pair-encoding (BPE) [18] to segment the code into tokens, while CoTEXT [19] opts for the Sentencepiece [20] model to extract tokens. These methods have been proven to be effective (see Fig. 1).

2.2. Graph based models

In the field of code vulnerability identification, GNN-based approaches leverage various code representations as input data, including AST, CFG, PDG, and CPG [5,21], and then employ graph embedding techniques or graph representation learning methods [22], such as Graph Convolutional Networks (GCNs), Graph Attention Networks (GATs), knowledge Graph [23]. These techniques aim to transform the complex, high-dimensional graph structures into compact, low-dimensional vector representations while preserving essential structural and semantic information [24–28]. Methods like AI4VA [29] and those proposed by Feng et al. [30] directly use the original versions of the four basic graphs as their code representations. The Devign [31] was the first to employ a GNN for code vulnerability detection tasks, incorporating Natural Code Sequence (NCS) edges into the CPG. Chakraborty et al. [32] proposed the ReVeal algorithm, which combines gated GNNs with multilayer perceptrons; FUNDED [10] introduced an enhanced AST with eight additional edge types. Unlike the aforementioned strategies that add structural information, VulSPG [33] suggests eliminating code unrelated to vulnerabilities. It performs graph slicing on the CPG to generate the SPG.

Sequence-based models excel at learning code semantics but struggle to capture the non-linear, hierarchical structures in code that involve control flow, nested functions, and variable dependencies. GNNs excel at learning from graph-structured data but are often constrained by single-layer structural information transfer. Moreover, when applied to code graphs, they struggle to incorporate the contextual semantic information of the original code. This insight motivated our proposal to combine pre-trained code language models, code graph models, and online knowledge distillation.

3. Vul-LMGNNs

In this section, we provide a detailed explanation of the Vul-LMGNNs process and its components. For clarity, our explanation is divided into several parts:

1. **Code Representation:** We utilize two perspectives for code representation:
   - Code graph representation
   - Code sequence representation

   We describe how to process these to produce the corresponding representations.
2. **Code Property Graph Creation:** We employ code property graphs to transform source code into an abstract graph representation. We introduce this code property graph here.
3. **Gated GNN:** For learning on the code property graph, we adopt a gated graph neural network. Its advantage lies in dynamically controlling information transmission and retention.
4. **Online Knowledge Distillation for Graph Neural Networks:** This part describes how we leverage online knowledge distillation to enable cross-layer knowledge transfer in the gated GNN, thereby improving its performance.
5. **Code Sequence Semantic Information Extraction:** Here, we use pre-trained code language models to extract contextual semantic embeddings from code sequences.
6. **Joint Training:** This section outlines how we jointly train the code language model and GNN to achieve implicit and explicit information fusion. This part will integrate the modules from Sections 3, 4, and 5.

Algorithm 1: Vul-LMGNNs: Code Vulnerability Detection

Input: Train data - D_{train}
Contribution of triplet loss - α
Contribution of regularization loss - β
Separation boundary - γ
Learning rate - lr
Tradeoff parameter - λ
Output: Trained model
Function `Vul-LMGNNs():`
  features $\leftarrow \emptyset$
  labels $\leftarrow \emptyset$
  ▷ Features extraction process
  for $(C, L) \in D_{train}$ **do**
    $(V, E) \leftarrow$ extract_code_property_graph(C)
    for $v \in V$ **do**
      $T_v \leftarrow$ onehot(v.type())
      $C_v, S_v \leftarrow$ *CodeLM*(v.fragment(), C)
      $x_v \leftarrow$ concat(T_v, C_v)
    end
    $\tilde{X} \leftarrow$ *Online-Distilled GGNN*(x_v, E)
    $x_g \leftarrow$ *Aggregate*($\tilde{X}$)
    features $\leftarrow$ features $\cup\, x_g \cup S_v$
    labels $\leftarrow$ labels $\cup\, L$
  end
  $M \leftarrow$ *Combined-RepresentationModel*()
  ▷ Model training process
  for $(f_g, l_g) \in D_{train}$ **do**
    ▷ Define the loss function.
    $L_{\text{all}} \leftarrow$ loss_function($M, D_{\text{train}}, f_g, l_g, \alpha, \beta, \gamma, \lambda$)
    θ represents the model parameters of $M_{Combined}$.
    $\theta \leftarrow \theta - \nabla_\theta(L_{\text{all}})$
  end
  return M_θ

3.1. Code representation

The purpose of this phase to transform the original function-level source code into fixed-length feature vectors that contains both semantic and syntactic structural information. Such conversion prepares the suitable data format for efficient processing by GNN models and code language models that follow. To achieve this, we adopt two specialized code representation strategies for GNNs and code sequence language models.

For code graph representation: We employ the open-source code analysis tool Joern [34] to parse the source code and generate the CPG. This CPG provides a unified and concise representation that combines control and data flow with abstract syntax trees and dependency graphs. We rigorously exclude functions with errors in graph generation to ensure data quality.

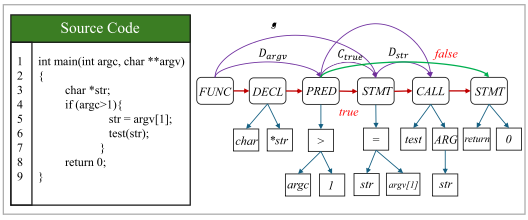

Fig. 2. A CPG for the example source code. Edge-type legend:*Blue = AST, Red = CFG, Purple = PDG*

For code sequence representation: we adhere to the approach presented in [31], by converting function-level code into natural code sequences. This method serializes the code in alignment with the natural order of the source code, thereby preserving the logical sequence of the code.

3.2. Code property graphs

In the process of generating CPG, functions are transformed into comprehensive graphs that comprise various types of nodes, such as variables and function calls, and edges, including control flow and data flow, which convey distinct types of information. At the core of the CPG, the AST captures the syntactic information, modeling the hierarchical structure of functions in a way that outlines the grammar and composition of the code. However, since the AST primarily offers a static representation, it lacks the capacity to infer the program's dynamic behavior. To address this, CPGs incorporate additional types of edges to represent data flow and control flow, thereby enriching the graph with insights into the execution context and dependencies between code segments. This integration frames a more holistic understanding of both the static structure and dynamic behavior of the program.

The representation of the CPG is denoted as $G = (V, E)$, where V represents the nodes within the graph and E means the edges. Each vertex V in the CPG encompasses the vertex type and a segment of the original code. As illustrated in Fig. 2, the nodes and blue edges represent the AST structure of this function segment, with the purple edges marked "D_{argv}" indicating the data dependency from the subtree defining variable $argv$ to the subtree using the defined value. The red edges denote the execution order within the function.

For the node set V, every node $v \in V$ can contain various types of information depending on its source, such as AST, CFG, or PDG. This includes CPG node type identifiers such as $IdentifierDeclType$ or keywords such as $int, char, for$, or operators such as $+, -$.

3.3. Gated graph neural network

In this section, we leverage GGNNs to explore CPGs, utilizing their advanced capabilities to discern patterns of information flow across nodes, thereby revealing structural insights pertinent to code properties.

GGNNs are fed with feature vectors of all the nodes alongside the graph edges. For a specified embedded graph $g_i(V, X, A)$, where V indicates nodes, X their features, and A the adjacency relationships, the GGNN assigns a Gated Recurrent Unit (GRU) to each node $v_j \in V$. This GRU updates the current vertex embedding by integrating the embeddings of all its neighboring nodes. Specifically, the initial state vector for a node $h_j^{(1)} \in \mathbb{R}^z$, where $z \geq d$, is initialized by copying x_j into the first dimensions and padding with additional zeros. To update node embeddings, we employ a neighborhood aggregation scheme. At each node, messages are aggregated and subsequently utilized to update the associated node representation at the subsequent embedding layer. Formally,

$$a_{v,g}^t = A_{(v,g)}^T \left(\left[h_1^{(t-1)T}, \ldots, h_m^{(t-1)T} \right] + b \right) \tag{1}$$

To be specific, t represents a specific time step, b denotes the bias vector, and A is the adjacency matrix. The subsequent state $a_{v,g}^t$ of node v_j is computed by aggregating the information from all neighboring nodes as defined in the adjacency matrix $A_{(v,g)}$ for a particular edge type.

Subsequently, the GRU algorithm is used to aggregate and update the states for identical nodes across different graphs. The process is articulated as follows:

$$z_{v,g}^t = \sigma(W^z \cdot AGG(a_{v,g}^t) + U^z h_{v,g}^{(t-1)}) \tag{2}$$

$$r_{v,g}^t = \sigma(W^r \cdot AGG(a_{v,g}^t) + U^r h_{v,g}^{(t-1)}) \tag{3}$$

$$\widetilde{h_{v,g}^t} = \tanh(W \cdot AGG(a_{v,g}^t) + U(r_{v,g}^t \circ h_{v,g}^{(t-1)})) \tag{4}$$

$$h_{v,g}^t = (1 - z_{v,g}^t) \circ h_{v,g}^{(t-1)} + z_{v,g}^t \circ \widetilde{h_{v,g}^t} \tag{5}$$

where $h_{v,g}^{(t-1)}$ is the hidden state of node v in graph g, $z_{v,g}^t$ and $r_{v,g}^t$ are the update gate and reset gate, respectively. $\hat{h}_{v,g}^t$ is the candidate hidden state, and $h_{v,g}^t$ is the output hidden state. AGG denotes the aggregation function, which is utilized to compile information from various edge types. In our application, we have employed the SUM [31] function.

The final step involves aggregating all vertex embeddings into a single vector to represent the entire CPG. Specifically,

$$H_{(v,g)}^{(T)} = \sum_{v \in V} h_{v,g}^t \tag{6}$$

Subsequently, we adopt a training mechanism similar to that of [4, 31], which deconstructs the task into 'learning code representation' and 'learning vulnerability'. This approach introduced an output layer designed to highlight the nodes with the most significant information for the task of vulnerability detection. We utilized convolution and max-pooling operations, commonly employed in CNNs. $\alpha(\cdot)$ is defined as a one-dimensional convolutional layer accompanied by max pooling, denoted as:

$$\alpha(\cdot) = MAXPOOL(Relu(CONV(\cdot))) \tag{7}$$

Given the total time steps T of the GGNN and the number of applications l of $\alpha(\cdot)$, the $Conv$ module is represented as:

$$Z_i^1 = \alpha([H_{(v,g)}^{(T)}, x_i]), \ldots, Z_i^{(l)} = \alpha(Z_i^{(l-1)}) \tag{8}$$

$$Y_i^{(1)} = \alpha(H_{(v,g)}^{(T)}), \ldots, Y_i^{(l)} = \alpha(Y_i^{(l-1)}) \tag{9}$$

where we apply 1-D convolutional and dense layers to $[H_{(v,g)}^{(T)}, x_i]$ and $H_{(v,g)}^{(T)}$. Afterward, we make a pairwise multiplication on the two outputs and make a prediction.

3.4. Online knowledge distillation for GGNNs

Knowledge Distillation (KD) is a technique in machine learning that transfers knowledge from a complex model (teacher) to a simpler model (student). Originally proposed for model compression, KD has evolved to become a powerful method for improving model performance across various domains. The methods of KD can be broadly categorized into three main approaches: (1) Soft Targets: This involves using the probability distributions over classes produced by the teacher model to train the student model to capture inter-class relationships . (2) Feature Embeddings: This approach focuses on transferring knowledge from the intermediate layers of the teacher model. It involves matching the feature representations or embeddings between the teacher and student models. This can be done through various methods such as minimizing

Algorithm 2: collaborative distillation

Input: The parameters θ_1 and θ_2 of the student S_1 and S_2; The calculated local structure by LSP module; hyper-parameter α and label y

Output: S_1 and S_2 with excellent performance.

Initialization parameters θ_1 and θ_2;
while *epochs* $\leq$ *max_epoch* **do**
  /* Training loss of S_1 */
  Obtain the structure preserving loss $\mathcal{L}_{str}^{S_1}$ with Equ. (4) and Equ. (5);
  Obtain the cross-entropy loss $\mathcal{L}_{ce}^{S_1}(y, p_1)$;
  Construct the total loss for S_1 as Equ. (7);
  /* Training loss of S_2 */
  Obtain the structure preserving loss $\mathcal{L}_{str}^{S_2}$ with Equ. (4) and Equ. (6);
  Obtain the cross-entropy loss: $\mathcal{L}_{ce}^{S_2}(y, p_2)$;
  Construct the total loss for S_2 as Equ. (7);
  /* Alternating training of S_1 and S_2 */
  Update the parameter θ_1 while keeping θ_2 fixed;
  Update the parameter θ_2 while keeping θ_1 fixed;
end

the L2 distance between feature maps or using specially designed transformation layers. (3) Structural Knowledge: This method aims to transfer the structural information learned by the teacher model. In the context of neural networks, this could involve mimicking the attention patterns in transformer models or preserving the relational information in graph neural networks. Techniques like Local Structure Preservation (LSP) in graph neural networks fall under this category.

Local Structure Preserving (LSP) is introduced as the first knowledge distillation module specifically designed for GNNs. Its purpose is to ensure that the local structure of the graph data learned by the student model closely resembles that of the teacher model, as referenced in [35,36]. The method calculates the similarity $L_{i,j}$ between each node i and its neighboring nodes j, where $(i, j) \in \epsilon$, in the feature space. This similarity is computed using one of several kernel functions and then normalized using a softmax function. The kernel functions for calculating the similarity $D(z_i, z_j)$ are given by the following equation:

$$D(z_i, z_j) = \begin{cases} \|z_i - z_j\|_2^2 & \text{Euclidean} \\ z_i \cdot z_j & \text{Linear} \\ (z_i \cdot z_j + c)^d & \text{Poly} \\ e^{-\frac{1}{2\sigma}\|z_i - z_j\|^2} & \text{RBF} \end{cases} \tag{10}$$

The similarity $L_{i,j}$ is further defined using a softmax-like normalization:

$$L_{i,j} = \frac{e^{D(z_i, z_j)}}{\sum_{j|(i,j)\in\epsilon}(e^{D(z_i, z_j)})}, \tag{11}$$

where z_i and z_j represent the node features. This equation normalizes the similarities, transforming them into a probability distribution.

The local structure L_i of node i is described as the probability distribution of similarities between node i and the nodes in its neighborhood.

To train the student model to emulate the local structure of the teacher model in the feature space, the Kullback–Leibler (KL) divergence is employed. The loss function for this LSP is given by:

$$\mathcal{L}_{lsp} = \frac{1}{N}\sum_{i=1}^{N} D_{KL}(L_i^T \parallel L_i^S). \tag{12}$$

LSP enables the transfer of local structural information from the teacher model to the student model. However, it requires a pre-trained teacher model. Inspired by this and [37], we use an idea of online knowledge distillation that does not require a pre-trained teacher model; instead, it relies on multiple student models learning from each other.

The core of this idea is a cross-layer knowledge distillation strategy, where one student layer is aligned with a layer at a different depth in another student model. During each round of alternate training, structure information from each student layer is transferred to the previous layer of another student model, eventually spreading the structure information across all layers.

In our work, we use two student models, S_1 and S_2, with identical GGNN architectures and H intermediate layers. They use $l_{i,j}^{S_1}$ and $l_{i,j}^{S_2}$ to represent the local structure of node j in the ith layer of S_1 and S_2 respectively.

For S_1, the local structure of its ith layer is required to match the $(i+1)$th layer of S_2, with the final layer H matching the first layer. The structure preserving loss is calculated as the sum of KL divergence losses over all layers:

$$\mathcal{L}_{str}^{S_1} = \sum_{i=1}^{H} L_{l_i}^{S_1}, \quad \mathcal{L}_{str}^{S_2} = \sum_{i=1}^{H} L_{l_i}^{S_2} \tag{13}$$

where

$$\mathcal{L}_{l_i}^{S_1} = \frac{1}{N}\sum_{j=1}^{N} D_{KL}(l_{i+1,j}^{S_2} \parallel l_{i,j}^{S_1}) \tag{14}$$

$$\mathcal{L}_{l_i}^{S_2} = \frac{1}{N}\sum_{j=1}^{N} D_{KL}(l_{i+1,j}^{S_1} \parallel l_{i,j}^{S_2}) \tag{15}$$

The total loss of S_1 and S_2 is formulated as:

$$\mathcal{L}_1 = \mathcal{L}_{ce}^{S_1}(y, p_1) + \alpha L_{str}^{S_1}, \mathcal{L}_2 = \mathcal{L}_{ce}^{S_2}(y, p_2) + \alpha L_{str}^{S_2} \tag{16}$$

where $\mathcal{L}_{ce}^{S_1/S_2}$ represents the cross entropy loss function, which is computed using the predictions (p_1 or p_2) from each student model and the true label y. The parameter α serves as a weighting factor, adjusting the relative importance of different loss components. As outlined in Algorithm 2, the training process alternates between the two student models, S_1 and S_2, allowing them to learn from each other iteratively.

If the student number is greater than 2, each model will capture local structure information from all remaining models. The ith layer's structure preserving loss of the kth model becomes

$$L_{l_i}^{S_k} = \frac{1}{M-1}\frac{1}{N}\sum_{p=1\vee p\neq k}^{M}\sum_{j=1}^{N} D_{KL}(l_{i+1,j}^{Sp} \parallel l_{i,j}^{S_k}) \tag{17}$$

where M is the number of student models.

Distinct from prior endeavors, our online knowledge distillation procedure interlinks code language models with Graph Neural Networks (GNNs). Consequently, it achieves not only the transference of structural knowledge across GNN layers but also facilitates the amalgamation of semantic insights from language models within the GNN framework.

3.5. Extracting code sequence semantics by CodeLMs

CodeLMs are designed to understand, generate, and manipulate programming code. Similar to natural language models [38], code language models are trained on vast corpora of source code from various programming languages and open-source repositories. They learn to recognize patterns, syntax, and semantics specific to programming languages, allowing them to perform a wide range of code-related tasks. Notable examples of code language models include: CodeBERT [15], GraphCodeBERT [16], CodeT5 [17], CuBERT [39], CodeGen [40] etc. These models typically use transformer-based architectures found in large language models. However, they are specifically fine-tuned on programming tasks and incorporate domain-specific knowledge about

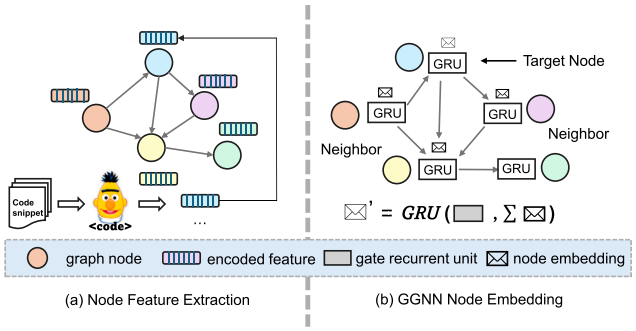

Fig. 3. Feature extraction and node embedding phases

code structure and best practices. By learning semantic code representations from massive codebases, CodeLMs have great potential for identifying potential security vulnerabilities in source code.

Now, we turn our attention to the handling of code sequences, which harbor a wealth of semantic information crucial for grasping the broader context and purpose of the code. Our approach to extracting this valuable semantic information begins with the transformation of function-level source code into code sequences. This conversion process is designed to preserve the logical flow that reflect the original order of declarations, operations, and control structures.

To process these code sequences, we fine-tune pre-trained codeLMs to capture the contextual information of various code elements, reveal implicit semantic features that may not be immediately apparent from the syntactic structure alone. We select several representative code language models. CodeBERT, with its bimodal pre-training in programming and natural language, excels in understanding the relationship between code and its natural language descriptions. GraphCodeBERT extends CodeBERT by incorporating code structure information. CodeT5, as a unified pre-trained encoder–decoder model, offers versatility in both comprehension and generation tasks, which can be particularly beneficial for more complex semantic analyses.

It is worth noting that our choice of code language model is deliberately flexible. We view these models as adaptable components in our larger framework, focusing on their utility in extracting semantic information rather than modifying the models themselves. This approach offers significant advantages: It allows easy updates as new, more advanced models become available.

3.6. Joint training of codelm and GNNs

We propose an "implicit-explicit" joint training framework to synergistically optimize both CodeLM and GNNs, harnessing the strengths of both models. Our method unfolds in two distinct phases:

1. Implicit Stage: We leverage CodeLM to initialize the node embeddings of the code GNN and simultaneously optimize both CodeLM and GNNs during the GNN's online knowledge distillation and label learning processes. This initialization process serves a dual purpose:
   (a) It facilitates the online knowledge distillation of the gated GNN, allowing for efficient transfer of knowledge from the language model to the graph structure.
   (b) It enhances the label learning process, providing a rich semantic foundation for the GNN to build upon.

   By using CodeLM for initialization and continuous optimization, we ensure that the GNN starts with and maintains a strong semantic understanding of the code, which is then refined through its graph-based processing.
2. Explicit Stage: This stage involves a more direct integration of the two models. We employ a linear interpolation technique to combine the outputs of CodeLM and the CodeLM-enhanced GNNs. This allows for a flexible, weighted combination of both models' predictions.

The implicit phase ensures that the GNN benefits from CodeLM's deep language understanding from the outset, while the explicit phase allows for a nuanced integration of both models' strengths in the final output.

Next, we provide a detailed description of the two stages.

3.6.1. Implicit stage

In our work, we utilize CodeLMs for initializing graph node embeddings. Specifically, we employ three different CodeLMs: CodeBERT [15], GraphCodeBERT, and CodeT5.For the sake of simplicity and clarity in our description, we will primarily focus on the process using CodeBERT as our exemplar CodeLM. The overall procedure, as depicted in Fig. 3, involves the following steps.

We start by decomposing the function into a sequence of statement sets $C = c_1, c_2, c_3, \ldots, c_n$, where each c_i is directly mapped to a node v_i within the CPG. This mapping ensures that the complex structure of a function is represented as an interconnected graph of simpler, manageable elements. Each statement set is tokenized using CodeBERT's pretrained Byte Pair Encoding (BPE) tokenizer [41], converting the statement into a series of tokens.

Following this, we initialize the self-embedding layer weights using CodeBERT's trained word embeddings for each token and employ label encoding for node type embeddings. In parallel, efforts are made to fine-tune CodeBERT on our target dataset, intending to tailor the model's understanding to our specific domain and thereby enhance the accuracy of the token vectorization process. Inspired by [29], we remove code properties from non-leaf nodes in the CPG, as these properties are often redundantly encoded in the leaf nodes. Finally, the node content embeddings derived from CodeBERT and the node type embeddings obtained through label encoding are concatenated to form a comprehensive initial representation for each node.

In our joint training approach, we optimize the parameters of both CodeLM and GGNN, leveraging the complementary strengths of each model — CodeLM's contextual understanding and GGNN's relational insights — to improve the model's performance in detecting code vulnerabilities. This joint optimization strategy is implemented through the use of cross-entropy loss across code graph nodes, allowing for the simultaneous optimization of parameters for CodeLM and GGNN. The formulated loss function can be depicted as:

$$L = -\sum_{c=1}^{M} y_{ic} \log(Softmax(MLP(Z_i^{(l)}) \odot MLP(Y_i^{(l)}))_{ic}) \tag{18}$$

M represents the number of classes, y_{ic} is a binary indicator (0 or 1) indicating whether class label c is the correct classification for observation i.

In this training process, CodeLM updates the node embeddings with each iteration, thereby gradually improving the complementary advantages of both CodeLM and GGNN.

3.6.2. Explicit stage

In the previous step, we implicitly combine CodeLM and GGNN by utilizing CodeLM to generate the node embeddings for GGNN. Here, we further explicitly combine the benefits of pre-training and graph-based approaches by leveraging interpolation predictions. Specifically, we introduce an auxiliary classifier that operates directly on CodeLM embeddings by feeding code embeddings E into a dense layer with softmax activation. Ultimately, we perform a linear interpolation [42] of the predictions from Vul-LMGNNs and CodeLM, which is expressed as follows:

$$Pred = \lambda Pred_{GGCN} + (1 - \lambda) \times Softmax(WE) \tag{19}$$

Table 1
Summary of datasets.

Dataset	#Vulnerable	#Non-Vul	Source	CWEs
DiverseVul	18,945	330,492	Snyk, Bugzilla	150
Devign	11,888	14,149	Github	N/A
VDSIC	82,411	119,1955	GitHub, Debian	4
ReVeal	1664	16,505	Chrome, Debian	N/A

The parameter λ controls the trade-off between the two objectives. A value of $\lambda = 1$ signifies the exclusive use of the full Vul-LMGNNs model, whereas $\lambda = 0$ indicates reliance solely on the CodeBERT module. When λ is within the range $(0, 1)$, it allows for a balanced integration of the predictions from both models. The fine-tuned CodeLM model regulated and optimized the input graph for the GGNN. Subsequently, an interpolation prediction facilitated an appropriate trade-off between the graph model and sequence model detection results, yielding outstanding detection outcomes.

4. Dataset review

To evaluate our proposed code vulnerability detection method and other baseline methods, it is imperative to possess a substantial quantity of both vulnerable and non-vulnerable source code, spanning a diverse range of vulnerabilities. In this paper, we have selected four public code vulnerability datasets, which include three widely-used popular datasets and one newly released comprehensive dataset. We have summarized the distribution of positive and negative samples and sources of the datasets, as well as whether they distinguish specific types of vulnerability, as shown in Table 1.

The DiverseVul [43] dataset is a newly released dataset of vulnerable source code. It has been curated by crawling two security issue websites that feature the most commits in git systems, extracting commits that fix vulnerabilities and the corresponding source codes from the projects. The dataset also employs deduplication of functions based on their MD5 hashes. This dataset comprises 18,945 vulnerable functions spanning over 150 CWEs, and 330,492 non-vulnerable functions extracted from 7,514 commits. The range of projects covered by this dataset exceeds the total of all previous datasets by 295. This dataset's substantial volume and diversity present a challenge for vulnerability detection methodologies.

The Devign dataset encompasses real-world function examples from GitHub, harvested from four renowned and diverse open-source libraries: Linux, FFmpeg, Qemu, and Wireshark. These examples are manually labeled based on commit messages and code differences. However, it does not provide information on the type of vulnerability or fine-grained labels. Additionally, this dataset is part of a programming language understanding evaluation benchmark known as CodeXGLUE [44], and has been extensively used by various methods.

The Draper VDISC dataset [4] is an extensive collection of 1.27 million functions extracted from open-source software, annotated with insights from three distinct static analyzers to flag potential vulnerabilities. It encompasses the four most common CWEs: CWE-120, CWE-119, CWE-469, and CWE-476. Notably, the dataset exhibits a highly imbalanced distribution of positive and negative samples, with a ratio nearing 1:14.5. This imbalance could adversely affect the real performance of our testing models. Therefore, in this paper, we have utilized a pre-processed version of the dataset with a more balanced distribution.

REVEAL [32] is a comprehensive real-world dataset, amassed by monitoring historical vulnerabilities from two prominent open-source projects: the Linux Debian Kernel and Chromium. It involves the extraction of the respective vulnerable and fixed versions of C/C++ source and header files that have been modified in patches, serving as positive and negative samples for research.

5. Experiments results

In this section, we present the experimental setup and the outcomes of our evaluations conducted on our proposed model, alongside six state-of-the-art baselines across four datasets. We have formulated the following four Research Questions (RQs) and have addressed them through our experimental investigations:

- **RQ1:** How does our Vul-LMGNNs performance compare with other learning-based methods for vulnerability identification?
- **RQ2:** With the variation of the trade-off parameter, what changes can be observed in the model's performance?
- **RQ3:** Is the fine-tuning process of pre-trained models a more efficient method for token vectorization in node embedding compared to initial word embedding weights?
- **RQ4:** How do different GNN architectures and pre-trained models influence the overall performance of the model?
- **RQ5:** Does online knowledge distillation enhance model performance? How does the number of student models alter performance?

The experiments were executed on a single NVIDIA A100 80 GB GPU. The system specifications comprised NVIDIA driver version 525.85.12 and CUDA version 11.8. The software environment was configured with Python 3.10.13 and torch 2.2.0.

In our comparative analysis, we evaluated the performance of Vul-LMGNN against 17 state-of-the-art detection models. This includes 5 Transformer-based models: BERT [45], CodeBERT [15], GraphCodeBERT [16], CodeT5-Small [17], and CodeT5-Base [17]. We also included 2 classical text-based techniques commonly used for vulnerability detection: TextGCN [46] and TextCNN [47]. In addition, we evaluated 10 specialized vulnerability detection algorithms: Devign [31], VulBERTa-MLP [48], VulBERTa-CNN [48], TokenCNN [4], VulDeePecker [8], SySeVR [14], CFExplainer [49], ReGVD [24], VulMAE [50], and ReVEAL [32]. For computational efficiency, functions with a node size exceeding 500 in the CPG were excluded from our analysis. In terms of our model's configuration, the learning rate and batch size were set to $1e{-}4$ and 64, respectively. The training was conducted over 20 epochs with an early stopping criterion triggered if no further optimization in performance. Specifically for the Devign model, AST was employed for the code graph representation.

To ensure fairness, we used official code repositories for baseline models when available and followed original papers closely where not. All models were trained under identical conditions, including hardware, dataset splits, batch sizes, and optimizers. Key parameters were set according to original recommendations or through grid search.

5.1. Comparison with baselines (RQ1)

To evaluate the performance of Vul-LMGNNs on code vulnerability detection, we executed an extensive comparative analysis against 17 baseline models utilizing the four datasets delineated in Table 1. The experimental results are systematically presented in Tables 2 and 3.

We initially tested the Vul-LMGNNs on datasets categorized by specific CWEs and analyzed their capability to recognize these CWEs within the test set. In terms of accuracy, precision, and F1 score, Vul-LMGNNs outperformed all baseline models. Specifically, any variant of Vul-LMGNNs outperformed all baseline models. Specifically, on the DiverseVul dataset, Vul-LMGNNs with CodeBERT as a component achieved an accuracy of 93.06% and an F1 score of 23.54%; on the balanced version of the VDSIC dataset, they reached an accuracy of 84.38%. Vul-LMGNNs with CodeT5 as a component achieved an accuracy of 95.04% on the DiverseVul dataset, with an F1 score of 32.59%, surpassing the previous best method by 10%.

Vul-LMGNNs outperform other models primarily due to their effective integration of structural information from GNNs and semantic

Table 2
Performance metrics of various models on datasets with specific CWEs.Note: C-B: CodeBERT, GC-B: GraphCodeBERT, C-T5s: CodeT5-Small, C-T5b: CodeT5-Base.

DiverseVul dataset					Draper VDSIC dataset				
Model	ACC (%)	P (%)	R (%)	F1 (%)	Model	ACC (%)	P (%)	R (%)	F1 (%)
Bert	91.99	27.95	13.09	17.83	Bert	79.41	81.86	75.97	78.80
CodeBert	92.40	28.26	20.02	23.44	CodeBert	83.13	86.13	78.97	82.39
GraphCodeBert	92.96	31.14	16.30	21.40	GraphCodeBert	83.98	84.74	83.17	83.95
CodeT5-Small	94.27	43.00	17.46	24.83	CodeT5-Small	84.47	84.21	84.70	84.51
CodeT5-Base	94.24	42.96	19.05	26.40	CodeT5-Base	85.01	85.90	84.64	85.12
TextCNN	92.16	10.25	9.82	10.03	TextCNN	66.54	65.36	70.55	67.86
TextGCN	91.50	15.66	11.50	13.27	TextGCN	67.55	67.66	67.63	67.64
Devign(AST)	70.21	9.35	9.22	9.28	Devign(AST)	59.30	58.84	68.93	63.49
VulBERTa-MLP	92.14	21.47	16.92	18.92	VulBERTa-MLP	80.41	83.37	75.97	79.50
VulBERTa-CNN	91.33	17.76	16.51	17.11	VulBERTa-CNN	78.98	79.46	78.17	78.81
TokenCNN	90.54	11.21	10.76	10.98	TokenCNN	65.05	63.99	68.84	66.33
VulDeePecker	91.76	17.31	13.76	15.33	VulDeePecker	78.15	78.50	77.53	78.01
SySeVR	92.86	27.10	18.76	22.17	SySeVR	82.01	83.25	80.15	81.67
CFExplainer	88.86	13.63	19.76	16.13	CFExplainer	80.01	83.37	74.97	78.95
ReGVD	85.86	10.26	20.16	13.73	ReGVD	79.78	80.15	79.17	79.66
VulMAE	88.78	11.21	15.46	13.00	VulMAE	77.05	76.12	78.84	77.45
ReVeal	85.88	9.35	18.46	12.42	ReVeal	78.15	78.50	77.53	78.01
Vul-LMGNNs(C-B)	**93.06**	**32.21**	**18.54**	**23.54**	Vul-LMGNNs(C-B)	**84.38**	**87.37**	**80.64**	**83.87**
Vul-LMGNNs(GC-B)	**93.61**	**33.73**	**18.51**	**23.90**	Vul-LMGNNs(GC-B)	**84.51**	**86.36**	**81.97**	**84.11**
Vul-LMGNNs(C-T5s)	**95.04**	**60.51**	**20.01**	**30.43**	Vul-LMGNNs(C-T5s)	**85.68**	**86.69**	**84.31**	**85.48**
Vul-LMGNNs(C-T5b)	**94.84**	**55.86**	**23.01**	**32.59**	Vul-LMGNNs(C-T5b)	**85.91**	**86.33**	**85.33**	**85.83**

Table 3
Performance metrics of various models on datasets with no specific CWEs.Note: C-B: CodeBERT, GC-B: GraphCodeBERT, C-T5s: CodeT5-Small, C-T5b: CodeT5-Base.

Devign dataset					ReVeal dataset				
Model	ACC (%)	P (%)	R (%)	F1 (%)	Model	ACC (%)	P (%)	R (%)	F1 (%)
Bert	60.58	57.67	54.64	56.11	Bert	86.88	32.70	40.13	36.04
CodeBert	63.93	61.03	58.10	59.53	CodeBert	88.64	38.26	38.13	38.19
GraphCodeBert	64.80	64.37	54.38	58.96	GraphCodeBert	89.25	41.67	41.81	41.74
CodeT5-Small	65.62	64.35	55.39	59.53	CodeT5-Small	90.50	47.81	40.73	43.99
CodeT5-Base	65.92	64.27	57.12	60.48	CodeT5-Base	90.94	50.69	39.76	44.56
TextCNN	60.38	59.03	57.72	58.37	TextCNN	85.43	26.32	20.33	22.94
TextGCN	60.47	60.87	58.58	59.70	TextGCN	87.25	24.61	17.85	20.69
Devign(AST)	57.66	56.96	56.25	56.60	Devign(AST)	87.49	36.65	31.55	33.91
VulBERTa-MLP	64.75	62.71	56.22	59.29	VulBERTa-MLP	88.48	36.79	35.90	36.34
VulBERTa-CNN	64.42	63.11	53.12	57.69	VulBERTa-CNN	87.64	34.46	38.76	36.48
TokenCNN	47.63	43.98	45.01	43.97	TokenCNN	73.48	17.47	50.90	26.01
VulDeePecker	60.36	56.94	55.41	56.39	VulDeePecker	89.05	17.68	13.87	15.70
SySeVR	62.18	60.45	54.05	57.30	SySeVR	84.22	24.36	40.11	30.25
CFExplainer	57.66	56.96	56.25	56.60	CFExplainer	85.18	24.21	29.01	26.39
ReGVD	61.89	60.74	48.20	53.75	ReGVD	90.63	64.70	14.47	23.65
VulMAE	62.36	60.47	50.71	55.16	VulMAE	88.24	30.25	21.76	25.31
ReVeal	62.38	59.80	53.71	56.59	ReVeal	85.37	29.90	40.91	33.87
Vul-LMGNNs(C-B)	**65.70**	**64.53**	**56.34**	**60.16**	Vul-LMGNNs(C-B)	**90.80**	**57.09**	**46.45**	**51.22**
Vul-LMGNNs(GC-B)	**66.33**	**64.73**	**57.77**	**61.01**	Vul-LMGNNs(GC-B)	**91.58**	**55.12**	**43.41**	**48.57**
Vul-LMGNNs(C-T5s)	**66.77**	**63.45**	**64.20**	**63.82**	Vul-LMGNNs(C-T5s)	**90.98**	**50.76**	**50.41**	**50.58**
Vul-LMGNNs(C-T5b)	**67.27**	**64.73**	**62.20**	**63.44**	Vul-LMGNNs(C-T5b)	**91.68**	**54.89**	**51.41**	**53.09**

understanding from pre-trained language models. While models like BERT and CodeBERT capture sequential semantics well, they lack the structural insights needed for vulnerability detection in complex codebases. In contrast, graph-based models like TextGCN struggle in our experiments, as they focus on word co-occurrence and miss critical code structure details. By leveraging both semantic and structural features, Vul-LMGNNs achieve superior performance across various metrics, excelling in both precision and recall when detecting vulnerabilities embedded in the code.

Furthermore, extensive experiments demonstrated that code language models trained on massive datasets generally outperform specialized vulnerability detection methods, despite not containing C/C++ programs in their pre-training datasets. Pre-trained LM like BERT, CodeBERT, and CodeT5 variants consistently achieved higher accuracy (91.99%–94.27%) compared to specialized methods such as VulDeePecker (91.76%), SySeVR (92.86%), and ReVeal (85.88%). CodeT5-Small and CodeT5-Base demonstrated particularly strong performance, benefiting from their larger architectural scale and pre-training, with accuracies of 94.27% and 94.24% respectively, and F1 scores of 24.83% and 26.40%. The other three datasets exhibitd similar trends. These results strongly suggest that the transfer learning capabilities and rich semantic understanding of large-scale pre-trained code language models provide a significant advantage in vulnerability detection tasks. Their ability to capture complex code patterns and contextualized representations appears to be more effective than the feature engineering approaches typically used in specialized vulnerability detection methods.

As shown in Fig. 5, we demonstrated the performance of our method on specific vulnerabilities. We compared the competitiveness of our approach with CodeBert and GraphCodeBert. In this comparison, our method is the Vul-LMGNNs with CodeBert as a component, which we refer to as CodeBertGGNN. Among the top 30 most frequently occurring CWEs in the test set, our model achieved the highest accuracy rate on 50% of CWEs. It can be observed that for some CWEs, the recognition accuracy of the model is generally low, such as CWE-310 (Cryptographic Issues) and CWE-189 (Numeric Errors), while for another subset of CWEs, there are high recognition accuracy rates, such

as CWE-134 (Controlled Format String) and CWE-770 (Allocation of Resources Without Limits or Throttling).

We analyzed the lower identification rates for CWE-310 and CWE-189 and identified several potential reasons for these challenges:

Regarding CWE-310 (Cryptographic Issues): These vulnerabilities primarily involve encryption problems, which can manifest in cryptographic algorithms, key management, or protocol implementations within the code. Vulnerability detection tools face difficulties in identifying CWE-310 class vulnerabilities for the following reasons: (1). While the model can learn some indicators of cryptographic vulnerabilities from contextual dependencies, certain encryption issues only become apparent under specific runtime conditions. (2). The complexity and subtlety of cryptographic implementations often require domain-specific knowledge that may be beyond the scope of general-purpose vulnerability detection tools.

Concerning CWE-189 (Numeric Errors): The challenges in detecting these vulnerabilities are similar to those of CWE-310: (1). Some numeric errors only manifest under specific input conditions or runtime environments, making it difficult for automated tools to capture these dynamic behaviors. (2). There is a significant human factor involved in CWE-189 vulnerabilities. Developers may introduce counting errors due to mistakes or oversights during coding, which are often subtle and not easily detectable by automated tools.

In both cases, while our model can infer some vulnerability indicators from code context and structure, it may struggle with issues that only become apparent during runtime or require deep domain-specific understanding.

In the realm of graph-based detection models, TextGCN, while performing well in text classification, showed mediocre results in code vulnerability detection experiments, achieving only a 91.50% accuracy rate on DiverseVul, with precision and recall at 15.66% and 11.50%, respectively. This may be due to TextGCN's focus on word co-occurrence, lacking structural code information. The AST version of Devign, which incorporates control flow and data flow information and uses Word2Vec along with the average of tokens for node vector representation, performed poorly with an accuracy of only 70.21%, a gap of 14.26%–23.31% from our F1 score. This could be attributed to the neglect of local semantic information of code within the node.

In vulnerability detection, false negatives (missed vulnerabilities) pose a significant risk. Vul-LMGNNs demonstrate strong recall across datasets, reducing false negatives compared to baseline models like Devign and ReVeal. On the ReVeal dataset, our Vul-LMGNN model achieved a recall of 51.41%, significantly higher than ReVeal's 40.91%. In security applications, lowering false negatives is critical, and our model's enhanced recall showcases its robustness in identifying vulnerabilities that other methods might miss. Additionally, we observed that false positives, although slightly higher in our model, are a reasonable trade-off for the improved vulnerability detection coverage.

The performance disparity on the other two datasets without specific CWEs is similar to those observed in the previous CWE-specific evaluations, as illustrated in Table 3. our proposed Vul-LMGNNs models across both the Devign and ReVeal datasets, showcasing significant improvements over existing approaches in vulnerability detection tasks. On the Devign dataset, Vul-LMGNNs(C-T5b) variant achieved the highest scores, with an accuracy of 67.27%, precision of 64.73%, recall of 62.20%, and F1 score of 63.44%. The performance gap is even more pronounced on the ReVeal dataset. Here, Vul-LMGNNs(C-T5b) attained remarkable results with an accuracy of 91.68%, precision of 54.89%, recall of 51.41%, and F1 score of 53.09%. The performance disparity between Vul-LMGNNs and other models is particularly evident when comparing to traditional approaches. For instance, on the ReVeal dataset, the best performing Vul-LMGNNs model achieved an F1 score that is nearly 17 percentage points higher than best baseline (53.09% vs. 36.48%).

In summary, the experimental results provide strong evidence for the effectiveness of Vul-LMGNNs in vulnerability detection tasks. The models' ability to capture both structural and semantic information in code results in improved recall, lower false negatives, and consistent outperformance across datasets. The models' ability to outperform existing state-of-the-art approaches across multiple metrics and datasets underscores their potential to significantly advance the field of automated code vulnerability detection.

Table 4
Performance across various node embedding and initialization methods.

Base	ACC(%)	P(%)	R(%)	F1(%)
CodeBERT	84.35	**87.75**	79.85	83.61
GraphCodeBert	84.05	86.46	80.74	83.50
Fine-tuned(C-B)	84.38	87.37	80.64	83.87
Fine-tuned(GC-B)	**84.51**	86.36	**81.97**	**84.11**

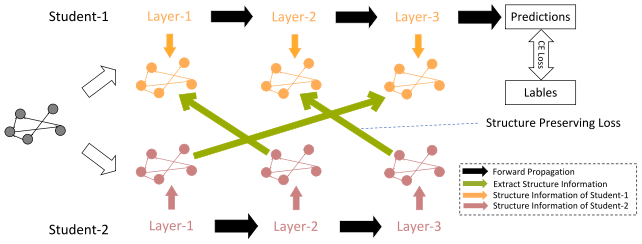

Fig. 4. Schematic diagram of online knowledge distillation based on knowledge alignment.

5.2. Impact of the tradeoff parameter (RQ2)

The parameter λ controls the trade-off between the training CodeLM and GNN. As λ approaches 1, the model's decisions rely more on the graph structure with the PL model embedding layer. Conversely, when λ approaches 0, the model leans toward sequence-based decisions. Our experiments using Vul-LMGNNs (C-B) across various datasets indicated that the optimal value of λ varies for different tasks, likely due to variations in vulnerability types and data distributions. For instance, on the partial Draper VDSIC dataset, increasing λ does not significantly improve model performance. This phenomenon can be attributed to the strong performance of sequence-based methods on the former VDSIC dataset.

Figs. 6 and 7 illustrated the evaluation matrix of Vul-LMGNNs with varying λ on the partial DiverseVul dataset. Accuracy improves consistently as λ increases, reaching its peak at $\lambda = 0.8$, slightly outperforming the GGNN or CodeBert alone (at $\lambda = 0$ or 1). The achieved accuracy is 90.24%. During this process, precision exhibits fluctuations but overall shows an upward trend, reaching 46. 19% at $\lambda = 0.8$, an improvement of 10. 71% over using the sequence model alone. However, recall follows a declining trend, reaching 36.45% at $\lambda = 0.8$, indicating that transformer-based PL models exhibit higher recall in certain vulnerability detection scenarios.

5.3. Evaluation of fine-tuning process (RQ3)

Pre-trained LMs demonstrate outstanding performance in various natural language processing tasks. Currently, there is a growing focus among researchers on employing pre-trained LMs for code-related tasks, including code search, code completion, code summarization and so on [17,51–53]. This has led to promising results in applications. This has prompted us to incorporate pre-trained models for programming languages in order to construct a novel vulnerability detection model.

We utilized the word embedding layer of pre-trained models as tokenization tools to generate node embeddings for graphs. These embeddings' weights are further fine-tuned during training. In our

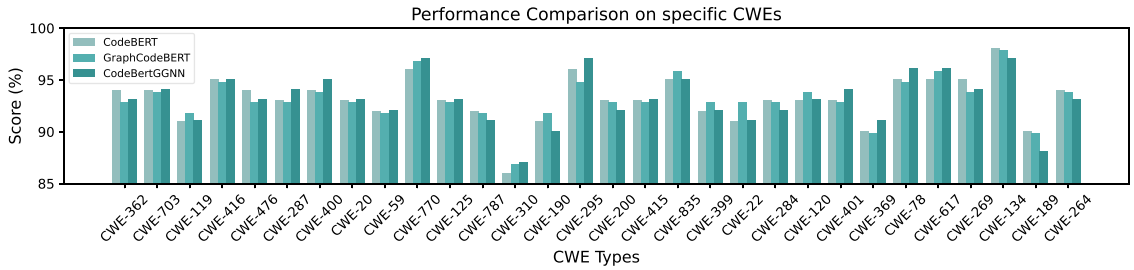

Fig. 5. Detection accuracy for the top-30 high-frequency CWE vulnerability types in DiverseVul. Note: CodeBertGGNN represents a basic variant of the proposed Vul-LMGNNs, specifically Vul-LMGNNs using CodeBERT as the Language Model (LM).

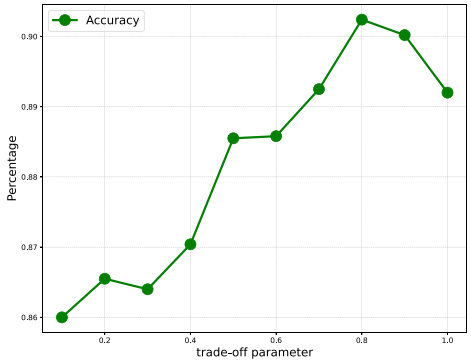

Fig. 6. Accuracy of Vul-LMGNNs when varying trade-off parameter on partial DiverseVul dataset.

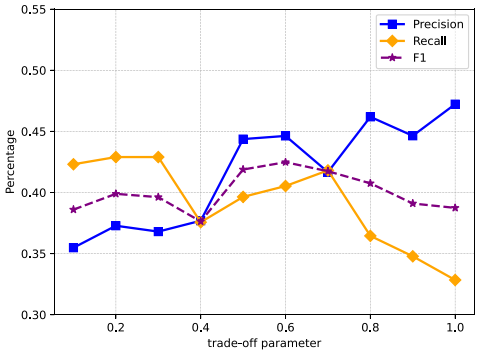

Fig. 7. Precision, recall and f1 of Vul-LMGNNs when varying trade-off parameter on partial DiverseVul dataset.

experiments, we explored three settings. First, we initialized our embedding layer weights using a fine-tuned CodeBERT which perform fine-tuning on the target dataset [54]. Additionally, we compared this approach with two others: initializing embedding layer weights directly using pre-trained CodeBERT and GraphCodeBERT, respectively. The results are summarized in Table 4.

The experimental results revealed that fine-tuning Code LMs models improves performance over using the base models for node embedding initialization. The base model CodeBERT achieved an accuracy of 84.35%, with an F1 score of 83.61%. When employing GraphCodeBERT, the accuracy slightly decreased to 84.05%, while F1 score improved to 83.50%. In contrast, fine-tuned CodeBERT (C-B) model achieved an accuracy of 84.38%, an F1 score of 83.87%. Notably, the fine-tuned GraphCodeBERT (GC-B) model outperformed all others with an accuracy of 84.51%, and an F1 score of 84.11%. These results suggested that fine-tuning the models leads to a better overall balance in performance metrics.

Table 5
Performance across various GNN architectures.

Combination	ACC(%)	P(%)	R(%)	F1(%)
GGNN+CodeBERT	**84.38**	**87.37**	**80.64**	**83.87**
GCN+CodeBERT	83.08	86.90	77.90	82.15
GAT+CodeBERT	79.29	81.92	75.15	78.39
SAGE-mean+CodeBERT	83.55	86.93	78.97	82.76
SAGE-GCN+CodeBERT	84.18	87.05	80.31	83.54
SAGE-pool+CodeBERT	81.90	85.82	76.43	80.85

5.4. Different GNN model combination (RQ4)

To investigate the impact of combining different GNNs with pre-trained LMs on vulnerability detection tasks, we compared distinct GNNs: Graph Gated Neural Network (GGNN), Graph Convolutional Network (GCN) [55], and Graph Attention Network (GAT) [56], and three variants of GraphSAGE (SAGE-mean, SAGE-GCN and SAGE-pool), all integrated with CodeBERT. For a fair comparison, we followed the configuration from [51], maintaining a consistent three-layer GNN architecture and setting GAT's number of heads to 8. Additionally, we employed fine-tuned CodeBERT with consistent model parameters. The specific experimental results are shown in Table 5.

As shown in Table 5, our experiments were conducted on the partial Draper VDSIC dataset. The results indicate that GGNN exhibit the best overall performance, with an accuracy of 84.38% and an F1 score of 83.87%. Compared to GGNN, GCN experiences decreases in accuracy and precision by 1.3% and 0.47%, respectively, with the most significant decrease observed in recall at 2.74%. This may be attributed to GCN treating all neighboring nodes equally during convolution, thus failing to assign different weights based on node importance, leading to inaccurate identification of nodes related to code vulnerabilities. Additionally, GCN updates node features for the entire graph in a single computation, which poses challenges when dealing with complex code graph structures in inductive learning tasks related to code vulnerabilities.

The performance of the GAT model exhibited a considerable gap compared to the previous two, with an accuracy of only 79.29%. Although GAT utilizes self-attention mechanisms to represent each node as a weighted sum of its neighbors, it does not fully leverage edge information, only utilizing connectivity, whereas edge information encompasses the control and data flow information of the code. In contrast, GGNN employs GRU units, allowing each node to receive messages from neighboring nodes at each iteration. This approach effectively captures both code data flow features and long sequence dependencies, resulting in outstanding performance (see Table 6).

5.5. Evaluation of online KD for GGNNs (RQ5)

Fig. 4 reveals the flow of structural information during the iterative training process of two GNNs in the online knowledge distillation approach. It can be observed that each layer of one student model aligns with the next layer of the other student model, with the final

Table 6
Ablation study for online knowledge distillation; "self" denotes the absence of online knowledge distillation.

KD Ablation	ACC(%)	P(%)	R(%)	F1(%)
Self	83.50	86.68	79.17	82.75
2 Student	**84.38**	87.37	80.64	83.87
3 Student	84.18	84.77	**83.33**	**84.04**
4 Student	83.90	87.93	78.59	83.00
5 Student	84.15	**88.93**	78.01	83.11
6 Student	83.61	86.22	80.01	83.00

Table 7
Comparison of experimental results for MixHop-based methods and KD-based methods on the partial draper VDSIC dataset.

Methods	ACC(%)	P(%)	R(%)	F1(%)
MixHop-based	83.68	87.60	78.47	82.78
KD-based	84.38	87.37	80.64	83.87

layer particularly matching the first layer. In this way, the two student models exchange structural information with each other, and propagate it across different layer depths. In this work, we evaluated two aspects of this approach: (1) the performance gain achieved and (2) how the performance changes as the number of student models is varied.

When transitioning from the "Self" baseline, which denotes the absence of online knowledge distillation, to using two student models, there is a significant enhancement across all measured metrics. This initial improvement underscores the benefit of the distillation process. Further analysis shows that the performance continues to improve with the addition of a third student model. However, this trend does not persist uniformly as the number of student models increases beyond three. For instance, while the metrics improve when moving from two to three student models, the subsequent addition of a fourth student model does not yield substantial gains and is followed by a slight decline in Accuracy and Precision. This indicates that, after reaching a certain threshold, the benefits of adding more student models begin to diminish.

5.6. Compared online KD with MixHop

In this paper, we leverage Online KD to address the limitations of single-layer information propagation in GNNs. This approach enables our method to effectively capture the intricate logic within abstract code graphs. While acknowledging the existence of algorithms like MixHop [57] in the GNN literature, which aim to capture long-range dependencies in graph neural networks, our experiment seeks to analyze and compare our KD-based method with MixHop.

We begin by examining the fundamental differences between these two approaches. MixHop is designed to overcome GNNs' limitations in handling long-distance relationships between nodes by mixing node features across multiple hop counts (i.e., neighborhood layers). In contrast, our Online Knowledge Distillation method focuses on transferring knowledge between teacher and student models. The key distinction is that online KD facilitates knowledge transfer between teacher and student models, whereas MixHop integrates node features across multiple hop counts. The implementation results are presented in the Table 7, providing a comparative analysis of these two approaches in addressing the challenges of information propagation and logic capture in GNNs for abstract code graphs.

Based on the experimental results, the Knowledge Distillation (KD) based method demonstrates superior performance compared to the MixHop-based approach on the Partial Draper VDSIC Dataset. The KD-based method achieves higher accuracy (84.38% vs. 83.68%) and significantly better recall (80.64% vs. 78.47%), indicating its enhanced ability to identify relevant instances. The KD-based approach's higher F1 score (83.87% vs. 82.78%) underscores its better overall balance between precision and recall. These results suggest that the KD-based method is more effective at capturing and utilizing relevant information from the dataset, making it the preferable choice.

6. Conclusion

In this paper, we propose a novel model, Vul-LMGNNs, which integrates sequence and graph embedding techniques to detect vulnerabilities in function-level source code. Our approach leverages the code property graph representation of the source code as the primary input. Specifically, we utilize a pre-trained Program Language (PL) model to extract local semantic features from the code, which are then embedded as nodes in the graph using sequence-based embeddings. Subsequently, we employ a GGNN equipped with convolutional layers to effectively fuse heterogeneous information within the graph. Finally, our model jointly learns and predicts vulnerabilities by combining the PL model with the GGNN. To validate the effectiveness of Vul-LMGNNs, we conducted extensive experiments on four real-world datasets, which demonstrated its superior performance. We systematically explored trade-off parameters, fine-tuning of the PL model, and variations of GNN architectures. Our findings further emphasize the positive contributions of each module to the overall model performance. As part of interesting future work, we intend to explore more effective fusion networks for learning code representations and facilitating multiclass detection.

CRediT authorship contribution statement

Ruitong Liu: Writing – original draft, Software, Methodology, Data curation. **Yanbin Wang:** Supervision, Methodology, Investigation. **Haitao Xu:** Funding acquisition, Formal analysis, Conceptualization. **Jianguo Sun:** Project administration. **Fan Zhang:** Visualization, Validation, Resources. **Peiyue Li:** Visualization, Validation. **Zhenhao Guo:** Writing – review & editing, Visualization, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Our source code could be accessed via the github: https://github.com/Vul-LMGNNs/vul-LMGGNN.

References

[1] Henrik Plate, Serena Elisa Ponta, Antonino Sabetta, Impact assessment for vulnerabilities in open-source software libraries, in: 2015 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2015, pp. 411–420.

[2] Guanjun Lin, Sheng Wen, Qing-Long Han, Jun Zhang, Yang Xiang, Software vulnerability detection using deep neural networks: a survey, Proc. IEEE 108 (10) (2020) 1825–1848.

[3] Stephan Lipp, Sebastian Banescu, Alexander Pretschner, An empirical study on the effectiveness of static C code analyzers for vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 544–555.

[4] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, Marc McConley, Automated vulnerability detection in source code using deep representation learning, in: 2018 17th IEEE International Conference on Machine Learning and Applications, ICMLA, IEEE, 2018, pp. 757–762.

[5] Bolun Wu, Futai Zou, et al., Code vulnerability detection based on deep sequence and graph models: A survey, Secur. Commun. Netw. 2022 (2022).

[6] Xu Nie, Ningke Li, Kailong Wang, Shangguang Wang, Xiapu Luo, Haoyu Wang, Understanding and tackling label errors in deep learning-based vulnerability detection (experience paper), in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, in: ISSTA 2023, Association for Computing Machinery, New York, NY, USA, 2023, pp. 52–63.

[7] Guanjun Lin, Jun Zhang, Wei Luo, Lei Pan, Olivier De Vel, Paul Montague, Yang Xiang, Software vulnerability discovery via learning multi-domain knowledge bases, IEEE Trans. Dependable Secure Comput. 18 (5) (2019) 2469–2485.

[8] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, Yuyi Zhong, Vuldeepecker: A deep learning-based system for vulnerability detection, 2018, arXiv preprint arXiv:1801.01681.

[9] Hongliang Liang, Yuxing Yang, Lu Sun, Lin Jiang, Jsac: A novel framework to detect malicious javascript via cnns over ast and cfg, in: 2019 International Joint Conference on Neural Networks, IJCNN, IEEE, 2019, pp. 1–8.

[10] Huanting Wang, Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Yansong Feng, Lizhong Bian, Zheng Wang, Combining graph-based learning with automated data collection for code vulnerability detection, IEEE Trans. Inf. Forensics Secur. 16 (2020) 1943–1958.

[11] Jacob A Harer, Louis Y Kim, Rebecca L Russell, Onur Ozdemir, Leonard R Kosta, Akshay Rangamani, Lei H Hamilton, Gabriel I Centeno, Jonathan R Key, Paul M Ellingwood, et al., Automated software vulnerability detection with machine learning, 2018, arXiv preprint arXiv:1803.04497.

[12] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, Hai Jin, μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection, IEEE Trans. Dependable Secure Comput. 18 (5) (2021) 2224–2236.

[13] Zhen Li, Deqing Zou, Shouhuai Xu, Zhaoxuan Chen, Yawei Zhu, Hai Jin, VulDeeLocator: A deep learning-based fine-grained vulnerability detector, IEEE Trans. Dependable Secure Comput. 19 (4) (2022) 2821–2837.

[14] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, Zhaoxuan Chen, Sysevr: A framework for using deep learning to detect software vulnerabilities, IEEE Trans. Dependable Secure Comput. 19 (4) (2021) 2244–2258.

[15] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al., Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint arXiv:2002.08155.

[16] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al., Graphcodebert: Pre-training code representations with data flow, 2020, arXiv preprint arXiv:2009.08366.

[17] Yue Wang, Weishi Wang, Shafiq Joty, Steven C.H. Hoi, Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, 2021, arXiv preprint arXiv:2109.00859.

[18] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al., Language models are unsupervised multitask learners, OpenAI blog 1 (8) (2019) 9.

[19] Long Phan, Hieu Tran, Daniel Le, Hieu Nguyen, James Anibal, Alec Peltekian, Yanfang Ye, Cotext: Multi-task learning with code-text transformer, 2021, arXiv preprint arXiv:2105.08645.

[20] Taku Kudo, John Richardson, Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing, 2018, arXiv preprint arXiv:1808.06226.

[21] Fabian Yamaguchi, Nico Golde, Daniel Arp, Konrad Rieck, Modeling and discovering vulnerabilities with code property graphs, in: 2014 IEEE Symposium on Security and Privacy, IEEE, 2014, pp. 590–604.

[22] Qiao Yuan, Sheng-Uei Guan, Pin Ni, Tianlun Luo, Ka Lok Man, Prudence Wong, Victor Chang, Continual graph learning: A survey, 2023, arXiv preprint arXiv:2301.12230.

[23] Pin Ni, Ramin Okhrati, Steven Guan, Victor Chang, Knowledge graph and deep learning-based text-to-GraphQL model for intelligent medical consultation chatbot, Inf. Syst. Front. 26 (1) (2024) 137–156.

[24] Van-Anh Nguyen, Dai Quoc Nguyen, Van Nguyen, Trung Le, Quan Hung Tran, Dinh Phung, ReGVD: Revisiting graph neural networks for vulnerability detection, in: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, 2022, pp. 178–182.

[25] Xiao Cheng, Guanqin Zhang, Haoyu Wang, Yulei Sui, Path-sensitive code embedding via contrastive learning for software vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 519–531.

[26] Yutao Hu, Suyuan Wang, Wenke Li, Junru Peng, Yueming Wu, Deqing Zou, Hai Jin, Interpreters for GNN-based vulnerability detection: Are we there yet? in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, in: ISSTA 2023, Association for Computing Machinery, New York, NY, USA, 2023, pp. 1407–1419.

[27] Yi Li, Shaohua Wang, Tien N. Nguyen, Vulnerability detection with fine-grained interpretations, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 292–303.

[28] David Hin, Andrey Kan, Huaming Chen, M. Ali Babar, LineVD: statement-level vulnerability detection using graph neural networks, in: Proceedings of the 19th International Conference on Mining Software Repositories, 2022, pp. 596–607.

[29] Sahil Suneja, Yunhui Zheng, Yufan Zhuang, Jim Laredo, Alessandro Morari, Learning to map source code to software vulnerability using code-as-a-graph, 2020, arXiv preprint arXiv:2006.08614.

[30] Qi Feng, Chendong Feng, Weijiang Hong, Graph neural network-based vulnerability predication, in: 2020 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2020, pp. 800–801.

[31] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, Yang Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, Adv. Neural Inf. Process. Syst. 32 (2019).

[32] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, Baishakhi Ray, Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9) (2021) 3280–3296.

[33] Weining Zheng, Yuan Jiang, Xiaohong Su, Vu1SPG: Vulnerability detection based on slice property graph representation learning, in: 2021 IEEE 32nd International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2021, pp. 457–467.

[34] Anon, The bug Hunter's workbench, 2023.

[35] Yiding Yang, Jiayan Qiu, Mingli Song, Dacheng Tao, Xinchao Wang, Distilling knowledge from graph convolutional networks, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 7074–7083.

[36] Jiongyu Guo, Defang Chen, Can Wang, Alignahead: online cross-layer knowledge extraction on graph neural networks, in: 2022 International Joint Conference on Neural Networks, IJCNN, IEEE, 2022, pp. 1–8.

[37] Jiongyu Guo, Defang Chen, Can Wang, Online cross-layer knowledge distillation on graph neural networks with deep supervision, Neural Comput. Appl. 35 (30) (2023) 22359–22374.

[38] Pin Ni, Gangmin Li, Patrick C.K. Hung, Victor Chang, StaResGRU-CNN with CMedLMs: A stacked residual GRU-CNN with pre-trained biomedical language models for predictive intelligence, Appl. Soft Comput. 113 (2021) 107975.

[39] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, Kensen Shi, Learning and evaluating contextual embedding of source code, in: International Conference on Machine Learning, PMLR, 2020, pp. 5110–5121.

[40] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, Caiming Xiong, Codegen: An open large language model for code with multi-turn program synthesis, 2022, arXiv preprint arXiv:2203.13474.

[41] Ali Araabi, Christof Monz, Vlad Niculae, How effective is byte pair encoding for out-of-vocabulary words in neural machine translation? 2022, arXiv preprint arXiv:2208.05225.

[42] Yuxiao Lin, Yuxian Meng, Xiaofei Sun, Qinghong Han, Kun Kuang, Jiwei Li, Fei Wu, Bertgcn: Transductive text classification by combining gcn and bert, 2021, arXiv preprint arXiv:2105.05727.

[43] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, David Wagner, Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, 2023, pp. 654–668.

[44] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al., Codexglue: A machine learning benchmark dataset for code understanding and generation, 2021, arXiv preprint arXiv:2102.04664.

[45] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv preprint arXiv:1810.04805.

[46] Liang Yao, Chengsheng Mao, Yuan Luo, Graph convolutional networks for text classification, in: Proceedings of the AAAI Conference on Artificial Intelligence, 33, (01) 2019, pp. 7370–7377.

[47] Bao Guo, Chunxia Zhang, Junmin Liu, Xiaoyi Ma, Improving text classification with weighted word embeddings via a multi-channel TextCNN model, Neurocomputing 363 (2019) 366–374.

[48] Hazim Hanif, Sergio Maffeis, Vulberta: Simplified source code pre-training for vulnerability detection, in: 2022 International Joint Conference on Neural Networks, IJCNN, IEEE, 2022, pp. 1–8.

[49] Fengyu Yang, Guangdong Zeng, Fa Zhong, Peng Xiao, Wei Zheng, Fuxing Qiu, CfExplainer: Explainable just-in-time defect prediction based on counterfactuals, J. Syst. Softw. (2024) 112182.

[50] Mahmoud Zamani, Saquib Irtiza, Latifur Khan, Kevin W Hamlen, VulMAE: Graph masked autoencoders for vulnerability detection from source and binary codes, in: International Symposium on Foundations and Practice of Security, Springer, 2023, pp. 191–207.

[51] Wei Tang, Mingwei Tang, Minchao Ban, Ziguo Zhao, Mingjun Feng, CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection, J. Syst. Softw. 199 (2023) 111623.

[52] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T Devanbu, Baishakhi Ray, Natgen: generative pre-training by "naturalizing" source code, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2022, pp. 18–30.

[53] Frank F Xu, Uri Alon, Graham Neubig, Vincent Josua Hellendoorn, A systematic evaluation of large language models of code, in: Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming, 2022, pp. 1–10.

[54] Ensheng Shi, Yanlin Wang, Hongyu Zhang, Lun Du, Shi Han, Dongmei Zhang, Hongbin Sun, Towards efficient fine-tuning of pre-trained code models: An experimental study and beyond, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, in: ISSTA 2023, Association for Computing Machinery, New York, NY, USA, 2023, pp. 39–51.

[55] Thomas N. Kipf, Max Welling, Semi-supervised classification with graph convolutional networks, 2016, arXiv preprint arXiv:1609.02907.

[56] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, Graph attention networks, 2017, arXiv preprint arXiv:1710.10903.

[57] Sami Abu-El-Haija, Bryan Perozzi, Amol Kapoor, Nazanin Alipourfard, Kristina Lerman, Hrayr Harutyunyan, Greg Ver Steeg, Aram Galstyan, Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing, in: International Conference on Machine Learning, PMLR, 2019, pp. 21–29.